

Snowflake Cortex on a Governed Data Foundation

Rebuilding the assistant inside the Data Cloud: dbt-modelled data, Cortex Search for documents and Cortex Analyst for metrics

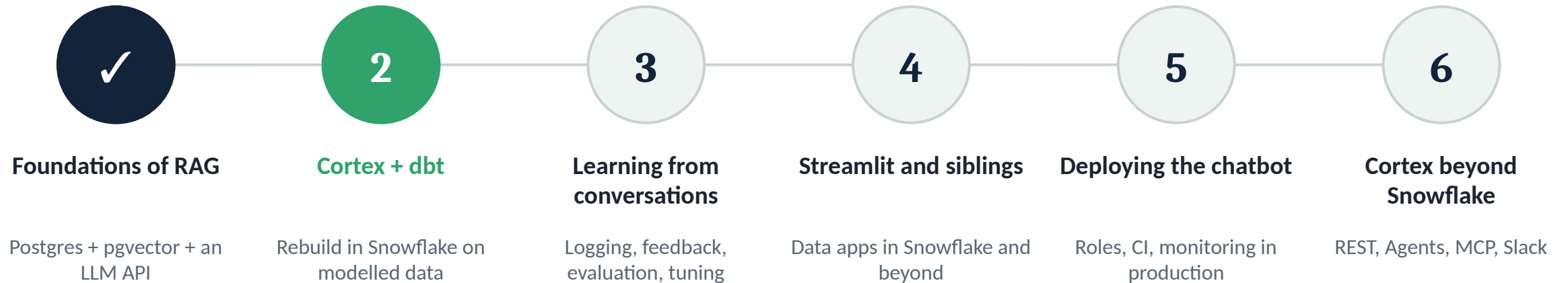
R. Firman Insan Muhammad (Vim)

SnowPro Certified • Data Engineering and AI Trainer

Scenario: Neo Nexus, a loyalty assistant for Caffè Nero. Illustrative only: all data, documents and figures are synthetic.



Same assistant, now inside the Data Cloud



Today's output: a dbt project, a Cortex Search service, a semantic view, and a recall@5 comparison against the Module 1 baseline.

By the end of today you can...



Map

each hand-built Module 1 component to its Snowflake Cortex equivalent, and explain what you give up.



Design

a dbt layer (staging, intermediate, marts) that makes both documents and tables AI-ready.



Build

a Cortex Search service on modelled chunks and a semantic view that Cortex Analyst can query.



Govern

the assistant with roles, masking and row access, and compare it fairly with the Module 1 baseline.

Everything we built by hand has a Snowflake equivalent

Module 1 component	Module 2 equivalent	Where it runs
PDF extraction in Python	AI_PARSE_DOCUMENT	dbt staging model
Character chunking	SPLIT_TEXT_RECURSIVE_CHARACTER	dbt intermediate model
Embedding model + pgvector HNSW	Cortex Search service	Serving layer
Vector search only	Hybrid: vector + keyword + semantic reranking	Built into Cortex Search
External LLM API call	AI_COMPLETE (Cortex LLM functions)	Inside Snowflake
<i>Not handled</i>	Cortex Analyst over a semantic view	Serving layer
Cron job re-runs ingest	TARGET_LAG + dbt schedule	Declarative

Green rows are new capabilities, not just replacements.

An LLM reads exactly what you give it

RAW.REWARDS_EXPORT

RWD_ID	RWD_STS	AMT	ISS_DT
RW-0091	R	3.40	20260411
RW-0091	R	3.40	20260411
RW-0107	I		11/04/2026

dbt

MARTS.FCT_REWARDS

reward_id	reward_status	reward_value	issued_date
RW-0091	Redeemed	3.40	2026-04-11
RW-0107	Issued	null	2026-04-11

Duplicates

double loads inflate every count

Cryptic codes

the model must guess that R means Redeemed

Mixed formats

dates and amounts parse inconsistently

No declared grain

one row per what? The LLM cannot know

Principle: the semantic layer an AI reads is a product. Model it, test it and document it like one.

01

The Cortex toolkit

What Snowflake manages for you: AI SQL functions, Cortex Search, Cortex Analyst and agents, all inside the governance boundary

Four capabilities on one governance foundation



AI SQL functions

AI_COMPLETE,
AI_PARSE_DOCUMENT,
AI_CLASSIFY, AI_EXTRACT,
AI_EMBED

*Row-level building blocks, callable
in dbt*



Cortex Search

Managed hybrid retrieval over
text with incremental refresh

Replaces Module 1's vector store



Cortex Analyst

Text-to-SQL grounded in a
semantic view

Answers the analytical questions



Cortex Agents

Plan, call Search and Analyst as
tools, and respond

Powers Snowflake Intelligence



Governance foundation: data stays in Snowflake; roles, masking and row access policies apply to every capability above.

One statement replaces Module 1's whole top lane

SNOWFLAKE SQL

```
CREATE OR REPLACE CORTEX SEARCH SERVICE
  serving.loyalty_search
  ON chunk_text
  ATTRIBUTES channel, doc_id
  WAREHOUSE = transform_wh
  TARGET_LAG = '1 hour'
  EMBEDDING_MODEL = 'snowflake-arctic-embed-l-v2.0'
AS
  SELECT chunk_id, doc_id, channel, chunk_text
  FROM marts.doc_chunks;
```

Requires change tracking on the source table.

ON

the text column that is searched

ATTRIBUTES

filterable columns, like last week's metadata WHERE clause

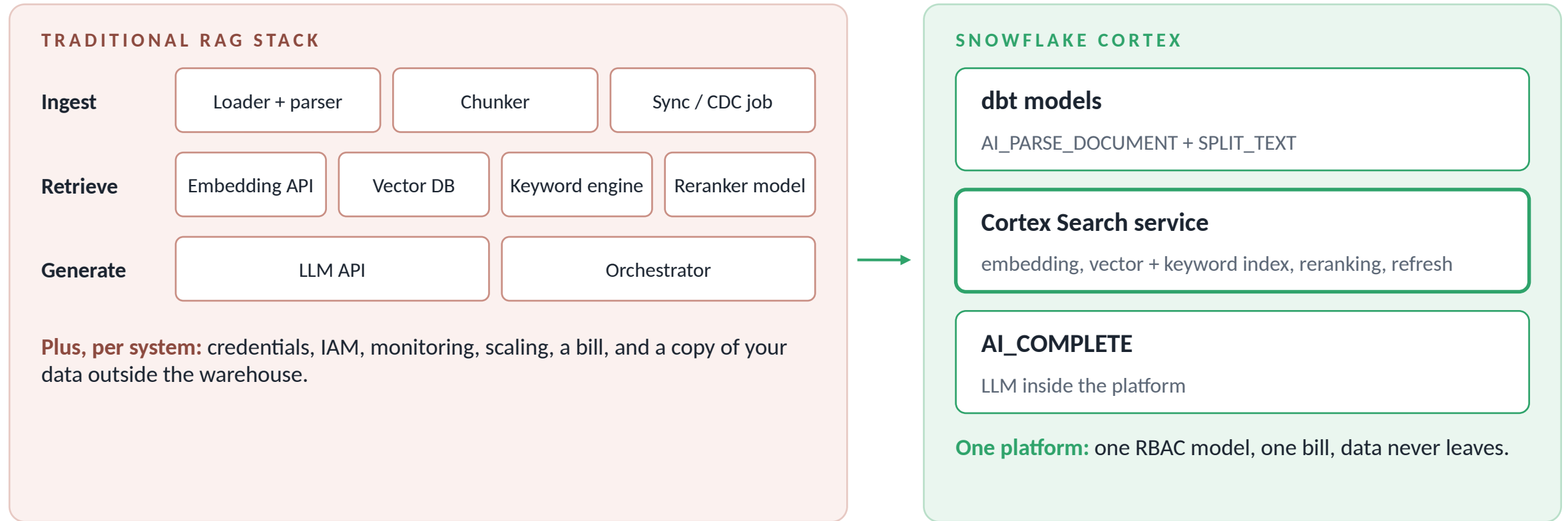
TARGET_LAG

declared freshness; incremental refresh

Hybrid by default

vector + keyword + semantic reranking

Nine moving parts collapse into three objects



9 → 3

moving parts to build and run

Many → 1

vendors, bills and security boundaries

Scripted → declared

freshness via TARGET_LAG

The operational burden moves to the platform

Task	Traditional RAG: you build and run it	Cortex: the platform does it
Keep index in sync with source	CDC job or cron, backfills, drift checks	TARGET_LAG incremental refresh
Embed chunks and queries identically	Same model in two code paths	Automatic, one model per service
Hybrid (vector + keyword) search	Run a second engine, fuse the scores	Built in
Reranking	Host and scale a cross-encoder	Built in
Scale and tune the index	Shard, tune HNSW, plan capacity	Managed serving
Change embedding model	Re-embed, dual-write, cut over	Recreate the service (still a full re-index)
Access control	Mirror permissions in every system	Snowflake roles on the service
Data residency	Text and vectors copied to vendors	Stays in Snowflake

Still your job: data modelling, chunking choices, prompt design and evaluation. Cortex simplifies infrastructure, not judgement.



Code, before and after

Seven stages of the same assistant, written the traditional way and the Cortex way, side by side

PDF parsing and chunking: a script becomes one query

TRADITIONAL RAG • PYTHON

16 lines

```
from pathlib import Path
from pypdf import PdfReader
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1800, chunk_overlap=300,
    separators=["\n## ", "\n\n", "\n", " "])

def parse_and_chunk(path, channel):
    reader = PdfReader(path)
    text = "\n".join(p.extract_text() or "" for p in reader.pages)
    text = strip_headers_footers(text) # your own helper
    return [{"doc_id": path.name, "channel": channel,
            "chunk_no": i, "chunk_text": c}
            for i, c in enumerate(splitter.split_text(text))]

for path in Path("loyalty_docs").rglob("*.pdf"):
    if path.stat().st_mtime > last_run_ts(): # incremental
        write_chunks(parse_and_chunk(path, path.parent.name))
```

CORTEX • SQL (DBT MODEL)

14 lines

```
select relative_path      as doc_id,
       split_part(relative_path,
                   '/', 1) as channel,
       c.index            as chunk_no,
       c.value::string    as chunk_text
from directory(@raw.loyalty_docs) d,
lateral flatten(input =>
  snowflake.cortex
    .split_text_recursive_character(
      ai_parse_document(
        to_file('@raw.loyalty_docs',
                d.relative_path),
        {'mode': 'LAYOUT'}):content::string,
        'markdown', 1800, 300)) c
```

Takeaway: no PDF library, no splitter dependency, no state file for incremental runs; dbt's incremental config replaces last_run_ts().

Embedding and two indexes: a pipeline becomes a declaration

TRADITIONAL RAG • PYTHON

17 lines

```
from sentence_transformers import SentenceTransformer
from elasticsearch import Elasticsearch, helpers

model = SentenceTransformer("BAAI/bge-small-en-v1.5")
es = Elasticsearch(ES_URL, api_key=ES_KEY)

def index_chunks(rows):
    vecs = model.encode([r["chunk_text"] for r in rows],
                        normalize_embeddings=True, batch_size=64)
    with pg.cursor() as cur:
        # vector index
        for r, v in zip(rows, vecs):
            cur.execute(UPSERT_SQL, (r["doc_id"], r["chunk_no"],
                                     r["chunk_text"], Jsonb(r), v))
    helpers.bulk(es, ({"_index": "loyalty_chunks", # keyword index
                      "_id": f'{r["doc_id"]}{r["chunk_no"]}',
                      "_source": r} for r in rows))

    pg.commit()

# not shown: HNSW DDL, ES mapping, orphan deletes,
# keeping both indexes consistent, re-embedding on model change
```

CORTEX • SQL

10 lines

```
create cortex search service
    serving.loyalty_search
    on chunk_text
    attributes channel, doc_id
    warehouse = transform_wh
    target_lag = '1 hour'
as
select chunk_id, doc_id,
       channel, chunk_text
from marts.doc_chunks;
```

Takeaway: one statement owns the embedding model, the vector index, the keyword index and keeping both in sync.

Hybrid retrieval and reranking: a module becomes a call

TRADITIONAL RAG • PYTHON

20 lines

```
from collections import defaultdict
from sentence_transformers import CrossEncoder
reranker = CrossEncoder("BAAI/bge-reranker-base")

def search(q, channel, k=5, n=50):
    qv = model.encode(q, normalize_embeddings=True)
    dense = pg.execute(DENSE_SQL,
                      (qv, channel, channel, qv, n)).fetchall()
    sparse = es.search(index="loyalty_chunks", size=n, query={
        "bool": {"must": {"match": {"chunk_text": q}},
        "filter": {"term": {"channel": channel}}}}
    )["hits"]["hits"]
    scores = defaultdict(float) # reciprocal rank fusion
    for rank, h in enumerate(dense):
        scores[h.chunk_id] += 1 / (60 + rank)
    for rank, h in enumerate(sparse):
        scores[h["_id"]] += 1 / (60 + rank)
    fused = sorted(scores, key=scores.get, reverse=True)[:n]
    texts = fetch_texts(fused)
    rr = reranker.predict([(q, t) for t in texts])
    return [c for _, c in sorted(zip(rr, fused), reverse=True)][:k]
```

CORTEX • PYTHON

8 lines

```
hits = svc.search(
    query=q,
    columns=["chunk_id",
             "chunk_text"],
    filter={"@eq":
            {"channel": channel}},
    limit=5,
).results
```

Takeaway: dense search, keyword search, fusion and reranking all happen server-side in one call.

Calling the LLM: a client with retries becomes a function

TRADITIONAL RAG • PYTHON

16 lines

```
import os, time
from openai import OpenAI, RateLimitError

client = OpenAI(api_key=os.environ["LLM_API_KEY"])

def generate(prompt, retries=3):
    for attempt in range(retries):
        try:
            r = client.chat.completions.create(
                model=LLM_MODEL,
                messages=[{"role": "user", "content": prompt}],
                temperature=0)
            return r.choices[0].message.content
        except RateLimitError:
            time.sleep(2 ** attempt)
    raise RuntimeError("LLM unavailable")

# not shown: secret rotation, egress approval,
# vendor DPA, per-vendor usage tracking
```

CORTEX • SQL

4 lines

```
select ai_complete(
    'claude-sonnet-4-5',
    :prompt
) as answer;
```

Takeaway: no API keys, no retry loop, no data leaving the platform; usage is metered with everything else.

Freshness: an orchestration DAG becomes one parameter

TRADITIONAL RAG • PYTHON

16 lines

```
from datetime import datetime
from airflow import DAG
from airflow.operators.python import PythonOperator

with DAG("loyalty_rag_refresh", schedule="@hourly",
        start_date=datetime(2026, 1, 1), catchup=False) as dag:
    detect = PythonOperator(task_id="detect_changed_docs",
                           python_callable=detect_changes)
    parse  = PythonOperator(task_id="parse_and_chunk",
                           python_callable=run_parse)
    embed  = PythonOperator(task_id="embed_and_index",
                           python_callable=run_index)
    prune  = PythonOperator(task_id="delete_orphans",
                           python_callable=prune_deleted)
    verify = PythonOperator(task_id="check_index_counts",
                           python_callable=verify_counts)
    detect >> parse >> embed >> prune >> verify
```

CORTEX • SQL

10 lines

```
-- on the search service
target_lag = '1 hour'

-- upstream dbt models, hourly
create task refresh_docs
  warehouse = transform_wh
  schedule  =
    'USING CRON 0 * * * * Europe/London'
as
  execute dbt project neo_nexus_dbt
    args = 'build --select +doc_chunks';
```

Takeaway: the index refreshes itself; you only schedule the dbt models that feed it.

Text-to-SQL: prompt engineering becomes a semantic contract

TRADITIONAL RAG • PYTHON

15 lines

```

SCHEMA = get_ddl_for(["fct_rewards", "dim_store"]) # dump DDL
RULES = """Redeemed rewards: reward_status = 'Redeemed'.
Join rewards to stores on store_id.
'Area' means region."""

def ask_sql(question):
    prompt = (f"{SCHEMA}\n{RULES}\nWrite one Snowflake SQL "
              f"query answering: {question}\nReturn SQL only.")
    sql = generate(prompt).strip("` \n")
    if not sql.lower().startswith("select"):
        raise ValueError("unsafe SQL")
    try:
        return sql, run_readonly(sql)
    except Exception as e:
        # self-repair loop
        sql = generate(f"{prompt}\nPrevious SQL failed: {e}")
        return sql, run_readonly(sql)

```

CORTEX • PYTHON

10 lines

```

r = requests.post(ANALYST_URL,
  headers=AUTH, json={
    "messages": [{"role": "user",
                  "content": [{"type": "text",
                                "text": question}]}],
    "semantic_view":
      "NEO_NEXUS.SERVING.REWARDS_SV"})

sql = next(c["statement"]
  for c in r.json()["message"]["content"]
  if c["type"] == "sql")

```

Takeaway: business rules move out of a prompt string into a governed semantic view that dbt can generate and test.

Access control: application logic becomes grants

TRADITIONAL RAG • PYTHON

15 lines

```

USER_REGIONS = load_entitlements()    # synced from IAM

def allowed_regions(user):
    regions = USER_REGIONS.get(user.id, [])
    if not regions:
        raise PermissionError("no entitlement")
    return regions

def secure_search(user, q):
    hits = search_any(q, allowed_regions(user))
    audit_log.write(user.id, q, [h.chunk_id for h in hits])
    return hits

def secure_sql(user, sql):
    sql = inject_row_filter(sql, allowed_regions(user))
    audit_log.write(user.id, sql)
    return run_readonly(sql)

# ...and again in the vector DB, the keyword engine and LLM logs

```

CORTEX • SQL

11 lines

```

create role loyalty_assistant;

grant usage on cortex search service
serving.loyalty_search
to role loyalty_assistant;

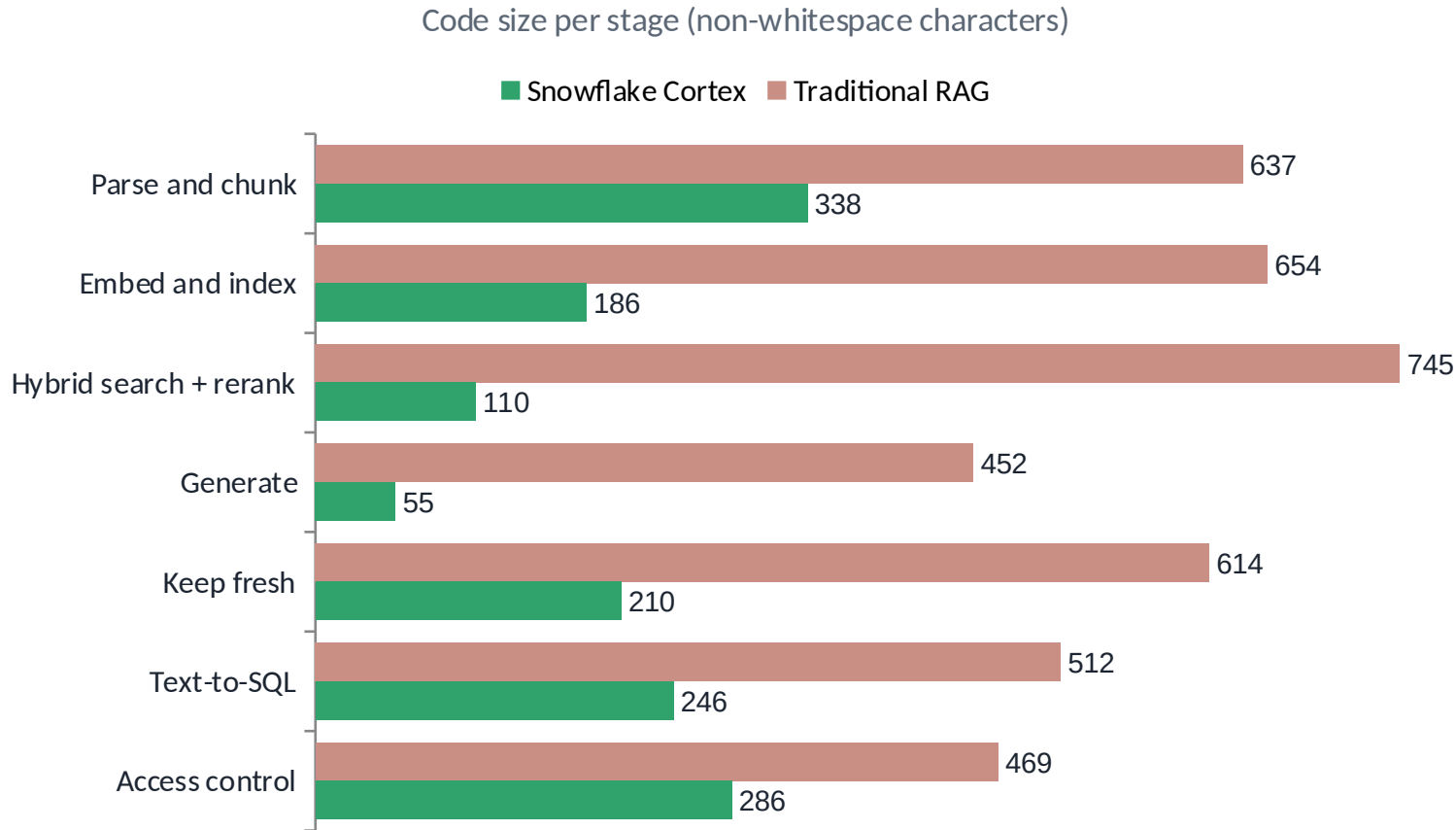
grant select on semantic view
serving.rewards_sv
to role loyalty_assistant;

-- Analyst SQL runs as the caller:
-- row access policies on the marts
-- apply automatically; queries are
-- in ACCESS_HISTORY for audit

```

Takeaway: permissions live once, in the platform, instead of being re-implemented in every system the data was copied to.

About two-thirds less code across the same seven stages

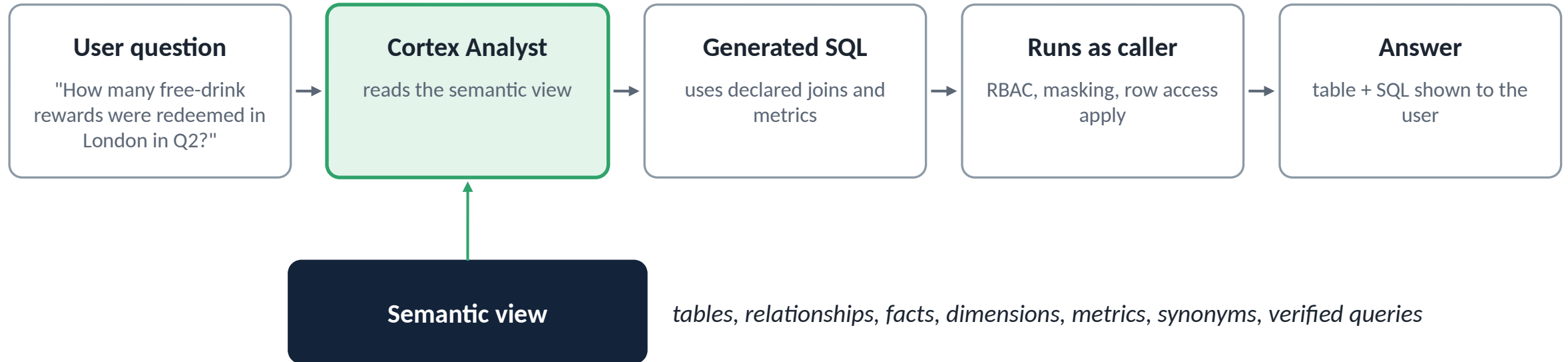


4,083 → 1,431

characters of code, about 65% less. In
lines: 115 → 67.

Not counted, traditional side: Elasticsearch,
Airflow, reranker hosting, secrets. **Not**
counted, Cortex side: the semantic view DDL.

Text-to-SQL that is grounded in business definitions



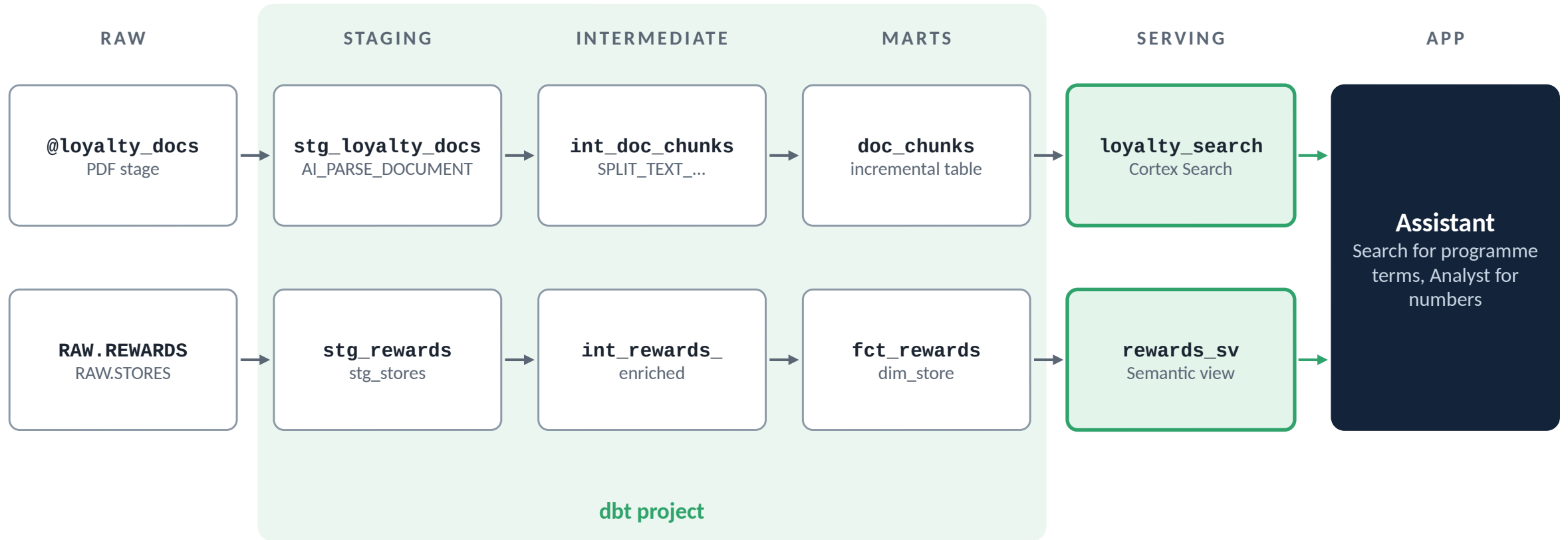
Why it beats Module 1 for numbers: it computes over every row instead of reading five similar ones, and it shows its SQL so answers are auditable.

02

dbt as the AI foundation

Staging, intermediate and mart models for both tables and documents, with Cortex services built on the marts

dbt owns the path from raw to AI-ready



Rule: the assistant reads marts only. Never RAW.

Organise the dbt project by layer and by purpose

REPOSITORY

```
neo_nexus_dbt/
├── models/
│   ├── staging/
│   │   ├── _sources.yml
│   │   ├── stg_rewards.sql
│   │   ├── stg_stores.sql
│   │   └── stg_loyalty_docs.sql
│   ├── intermediate/
│   │   ├── int_rewards_enriched.sql
│   │   └── int_doc_chunks.sql
│   └── marts/
│       ├── _marts.yml      # tests + docs
│       ├── fct_rewards.sql
│       ├── dim_store.sql
│       └── doc_chunks.sql
├── macros/
│   ├── create_search_service.sql
│   └── create_semantic_view.sql
└── dbt_project.yml
```



Staging

1:1 with sources. Rename, cast, decode codes. No joins.



Intermediate

Joins, enrichment, and document parsing into chunks.



Marts

The contract with consumers. The AI is now one of them.



Macros

Create Cortex objects, versioned beside their models.

Staging decodes; tests prove it

DBT MODEL

```
-- models/staging/stg_rewards.sql
with src as (
  select * from {{ source('raw', 'rewards_export') }}
  qualify row_number() over (
    partition by rwd_id order by _loaded_at desc) = 1
)
select
  rwd_id                as reward_id,
  store_no              as store_id,
  decode(rwd_sts, 'R', 'Redeemed', 'I', 'Issued',
    'E', 'Expired', 'C', 'Cancelled')
    as reward_status,
  try_to_number(amt, 12, 2) as reward_value,
  coalesce(try_to_date(iss_dt, 'YYYYMMDD'),
    try_to_date(iss_dt, 'DD/MM/YYYY'))
    as issued_date
from src
```

DBT TESTS + DOCS

```
# models/staging/_staging.yml
models:
  - name: stg_rewards
    description: One row per reward.
    columns:
      - name: reward_id
        tests: [unique, not_null]
      - name: reward_status
        description: Final status.
        tests:
          - accepted_values:
              values: [Redeemed, Issued,
                Expired, Cancelled]
      - name: issued_date
        tests: [not_null]
```


Parsing and chunking become SQL models

PARSE

```
-- models/staging/stg_loyalty_docs.sql
{{ config(materialized='incremental',
    unique_key='doc_id') }}

select
    relative_path                as doc_id,
    split_part(relative_path, '/', 1) as channel,
    last_modified,
    ai_parse_document(
        to_file('@raw.loyalty_docs', relative_path),
        {'mode': 'LAYOUT'})
    ):content::string            as doc_text
from directory(@raw.loyalty_docs)
{% if is_incremental() %}
where last_modified >
    (select max(last_modified) from {{ this }})
{% endif %}
```

CHUNK

```
-- models/intermediate/int_doc_chunks.sql
select
    d.doc_id,
    d.channel,
    c.index                as chunk_no,
    d.doc_id || '#' || c.index as chunk_id,
    c.value::string        as chunk_text
from {{ ref('stg_loyalty_docs') }} d,
lateral flatten(
    input => snowflake.cortex
        .split_text_recursive_character(
            d.doc_text,
            'markdown', -- respect headings
            1800,       -- chunk size (chars)
            300)        -- overlap
) c
```

Create the search service from dbt, next to its source

MART

```
-- models/marts/doc_chunks.sql
{{ config(
    materialized = 'incremental',
    unique_key   = 'chunk_id',
    post_hook    = [
        "alter table {{ this }}"
        "set change_tracking = true",
        "{{ create_search_service(this) }}"
    ]
) }}
select chunk_id, doc_id, channel,
       chunk_no, chunk_text
from {{ ref('int_doc_chunks') }}
```

MACRO

```
-- macros/create_search_service.sql
{% macro create_search_service(rel) %}
create cortex search service if not exists
    {{ rel.database }}.serving.loyalty_search
on chunk_text
attributes channel, doc_id
warehouse = transform_wh
target_lag = '1 hour'
as
select chunk_id, doc_id,
       channel, chunk_text
from {{ rel }}
{% endmacro %}
```

Declare the business meaning once, on the marts

SNOWFLAKE SQL

```
create or replace semantic view serving.rewards_sv
  tables (
    rewards  as marts.fct_rewards primary key (reward_id)
              comment = 'One row per reward issued',
    stores   as marts.dim_store primary key (store_id)
  )
  relationships (
    reward_store as rewards (store_id) references stores
  )
  facts (
    rewards.reward_value as reward_value
  )
  dimensions (
    stores.region as region
    with synonyms = ('area', 'city'),
    rewards.reward_status as reward_status,
    rewards.issued_date as issued_date
  )
  metrics (
    rewards.reward_count as count(rewards.reward_id),
    rewards.total_value as sum(rewards.reward_value)
  );
```

Tables

marts only, with primary keys

Relationships

joins declared, never inferred

Synonyms

how people talk, mapped to columns

Metrics

one formula per KPI, for everyone

03

The assistant

Answering from documents and from tables, routing between them, and keeping it governed

The Module 1 answer function, rebuilt on Cortex

PYTHON • SNOWPARK

```
from snowflake.core import Root
from snowflake.cortex import complete

svc = (Root(session).databases["NEO_NEXUS"]
       .schemas["SERVING"].cortex_search_services["LOYALTY_SEARCH"])

def answer(question, channel=None):
    kw = {"filter": {"@eq": {"channel": channel}}} if channel else {}
    hits = svc.search(query=question,
                      columns=["chunk_id", "chunk_text"],
                      limit=5, **kw).results
    context = "\n\n".join(
        f"[{h['chunk_id']}] {h['chunk_text']}" for h in hits)
    prompt = (f"{SYSTEM}\n\nContext: \n{context}"
              f"\n\nQuestion: {question}")
    return complete("claude-sonnet-4-5", prompt,
                   session=session), hits
```

What changed

- No embedding model to load
- No similarity SQL to tune
- Hybrid search and reranking included
- LLM runs inside Snowflake

What stayed

The prompt, the citation contract and the abstain rule: your design, not the vendor's.

One function, many engines: change a string, not a stack

Family	Example AI_COMPLETE model strings
Anthropic Claude	claude-opus-5-5, claude-sonnet-4-5
OpenAI GPT	openai-gpt-5.5, openai-gpt-5-mini
Meta Llama	llama3.3-70b, llama3.1-8b
Mistral	mistral-large3
Google Gemini	gemini-3.1-pro

Regional availability varies; some models need cross-region inference.

Module 1: new engine = new SDK, key and infrastructure. **Cortex:** new engine = new string.

CORTEX · SQL

```
-- same prompts, three engines,  
-- one query: a model bake-off  
select  
    question_id,  
    ai_complete('claude-sonnet-4-5',  
                prompt) as claude,  
    ai_complete('openai-gpt-5-mini',  
                prompt) as gpt,  
    ai_complete('llama3.3-70b',  
                prompt) as llama  
from eval.golden_prompts;
```

Pick the engine per task, and govern which engines are allowed

1

Route: classify DOCUMENT vs ANALYTICAL

claude-haiku-4-5 · llama3.1-8b

2

Answer: grounded response with citations

claude-sonnet-4-5 · openai-gpt-5-mini

3

Judge: score answers in evaluation (Module 3)

claude-opus-5-5 · openai-gpt-5.5

GOVERNANCE • SQL + PYTHON

```
-- admin: only approved engines
alter account set cortex_models_allowlist =
  'claude-sonnet-4-5,claude-haiku-4-5,llama3.3-70b';

-- admin: allow other regions (residency call)
alter account set cortex_enabled_cross_region =
  'AWS_EU';

# Module 1's OpenAI-style client, pointed at Cortex
client = OpenAI(
  api_key=SNOWFLAKE_PAT,
  base_url=f"https://{ACCOUNT}.snowflakecomputing.com"
    "/api/v2/cortex/v1")
client.chat.completions.create(
  model="claude-sonnet-4-5", messages=msgs)
```

Two routes, one assistant

CORTEX ANALYST · REST

```
resp = requests.post(
    f"https://{ACCOUNT}.snowflakecomputing.com"
    "/api/v2/cortex/analyst/message",
    headers={"Authorization": f"Bearer {TOKEN}",
            "Content-Type": "application/json"},
    json={"messages": [{"role": "user", "content": [
        {"type": "text", "text": question}]}],
        "semantic_view": "NEO_NEXUS.SERVING.REWARDS_SV"})

content = resp.json()["message"]["content"]
sql = next(c["statement"] for c in content
           if c["type"] == "sql")
df = session.sql(sql).to_pandas()  # runs as the user
```

1

Classifier prompt

AI_COMPLETE labels the question: DOCUMENT or ANALYTICAL. Transparent and easy to test.

2

Cortex Agent

Search and Analyst registered as tools; the agent plans and calls them. Covered in Module 6.

The assistant inherits Snowflake's security model



Least-privilege role

USAGE on the search service, SELECT on marts and the semantic view. Nothing on RAW.



Policies still apply

Analyst SQL runs as the caller: row access and masking policies filter every answer.



Data stays in place

Models run inside Snowflake. Cross-region inference is an explicit admin setting.



Cost is observable

Refresh warehouse, serving and tokens are metered and visible in ACCOUNT_USAGE.

What we gained, and what we gave up

Dimension	Module 1: DIY	Module 2: Cortex + dbt
Code to maintain	Ingest, index, retrieval, LLM client	SQL models, one service, one view
Retrieval	Vector only (hybrid as stretch)	Hybrid + reranking built in
Analytical questions	Not supported	Cortex Analyst
Freshness	Re-run a script	Declared: TARGET_LAG, dbt schedule
Governance	Application code	Snowflake RBAC and policies
Control over internals	Full	Limited: managed embeddings and reranker
Portability	Any Postgres, any LLM	Snowflake-specific

Decide with evidence: the lab compares recall@5 on the same golden set.

Rebuild the Neo Nexus assistant in Snowflake

- | | | |
|----------|--|--------|
| 1 | Set up
load raw tables; upload PDFs to @raw.loyalty_docs | 10 min |
| 2 | Model tables
stg, int and marts for rewards and stores, with tests | 25 min |
| 3 | Model documents
parse, chunk, doc_chunks, search service | 20 min |
| 4 | Semantic view
rewards_sv; ask five analytical questions | 15 min |
| 5 | Compare
rerun the Module 1 golden set; recall@5 old vs new | 20 min |

Deliverables

- dbt project: all models build, all tests pass
- loyalty_search service and rewards_sv semantic view
- Recall@5: Module 1 vs Module 2 on the same questions
- Half-page note: which change moved recall most, and why

Stretch: verified queries for the top three analytical questions

Questions worth arguing about

1. Where should a metric's definition live: dbt, the semantic view, or both? How do you stop them drifting?
2. Cortex Search hides its embedding and reranking choices. What do you lose, and how would you detect a regression?
3. Is there ever a good reason to let an assistant read RAW?
4. Who should own the semantic view: data engineering, analytics or the business?



References

Snowflake docs: Cortex Search overview; CREATE CORTEX SEARCH SERVICE

Snowflake docs: Semantic views; Cortex Analyst REST API

Snowflake docs: AI_PARSE_DOCUMENT; SPLIT_TEXT_RECURSIVE_CHARACTER

dbt Labs: How we structure our dbt projects

Kimball and Ross (2013) The Data Warehouse Toolkit, 3rd ed. Wiley.

SnowPro prep: Cortex and AI topics in the current SnowPro Core study guide

Learning from conversations



Log every turn

Questions, retrieved chunks, SQL, answers and latency, stored in Snowflake



Capture feedback

Thumbs, corrections and abandonment as training signal



Evaluate and tune

Golden sets, LLM-as-judge, prompt and retrieval tuning, cost

Before Module 3: finish the lab, keep your recall@5 results, and add 10 new questions from a colleague who has never seen the data.